

**Mobile Bildverarbeitung zur
Fehlererkennung und
-klassifizierung in der Produktion**

Noel Sappeck | sk237 Pötsch



Gliederung



01 **Einleitung**

02 **Theoretische Grundlage**

03 **Methodisches Vorgehen**

04 **Probleme**

05 **Ergebnisse**



01

Einleitung

Inwieweit eignet sich ein mobiles Bildverarbeitungssystem zur automatisierten Fehlererkennung in der Produktion?

Bedeutung der Qualitätssicherung

x



- Vermeidung von Ausschuss und Produktionskosten
- Sicherstellung gleichbleibender Produktqualität
- Einhaltung von Normen und Qualitätsstandards
- Vertrauen und Image bei Kunden

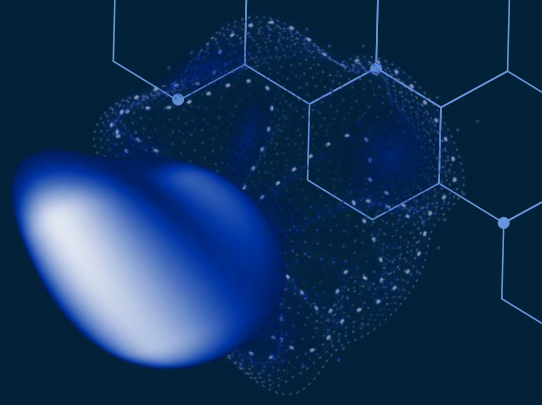
x

x



Gründe für Bildverarbeitungssysteme

- Schnelle Analyse großer Produktmengen
- Hohe und konstante Erkennungsgenauigkeit
- Objektive und reproduzierbare Bewertung
- Möglichkeit der 100 %-Kontrolle aller Teile



x

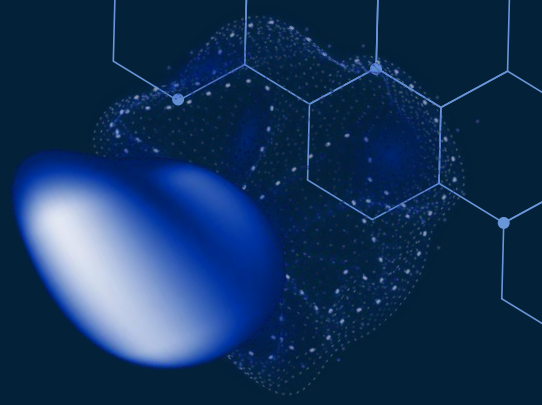
Grenzen der manuellen Qualitätskontrolle

- Zeitaufwendig bei großen Stückzahlen
- Fehleranfällig durch Ermüdung des Menschen
- Subjektive Bewertung möglich
- Begrenzte Geschwindigkeit der Kontrolle

x



x



x





x

Ziel der Arbeit



- Entwicklung eines mobilen Bildverarbeitungssystems
- Automatische Erkennung von Produktionsfehlern
- Klassifizierung von Bauteilen (OK / Fehler)
- Bewertung der Eignung für den Produktionseinsatz

x



x





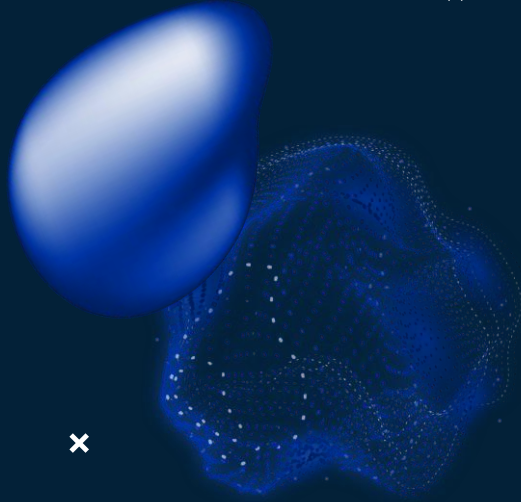
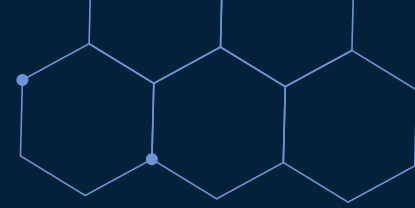
02

Theoretische Grundlage

*Darstellung der Grundlagen von Qualitätssicherung
und Bildverarbeitung*

Fehler in der Produktion

x



x

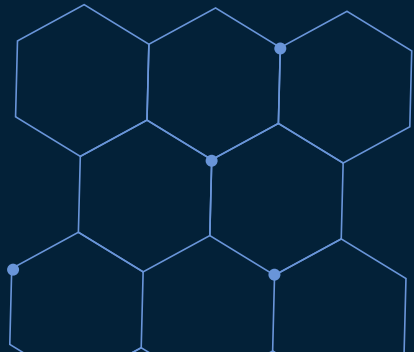
Welche Fehler passieren häufig?

- Fehlendes Material
- Verfärbung
- Kratzer
- Lufteinschlüsse
- Verformungen

x

→ welche Fehler sind für das Projekt machbar?

x



Grundlagen der Bildverarbeitung

Text



- Aufnahme von Bildern durch Kameras
- Verarbeitung der Bilddaten durch Software
- Analyse von Merkmalen wie Form, Farbe oder Struktur
- Nutzung zur automatischen Qualitätskontrolle



Objekterkennung mit KI

×

- Lokalisierung von Objekten im Bild
- Einordnung in verschiedene Klassen
- Darstellung durch Bounding Boxes
- Anwendung in Industrie, Verkehr oder Medizin

×

×

×

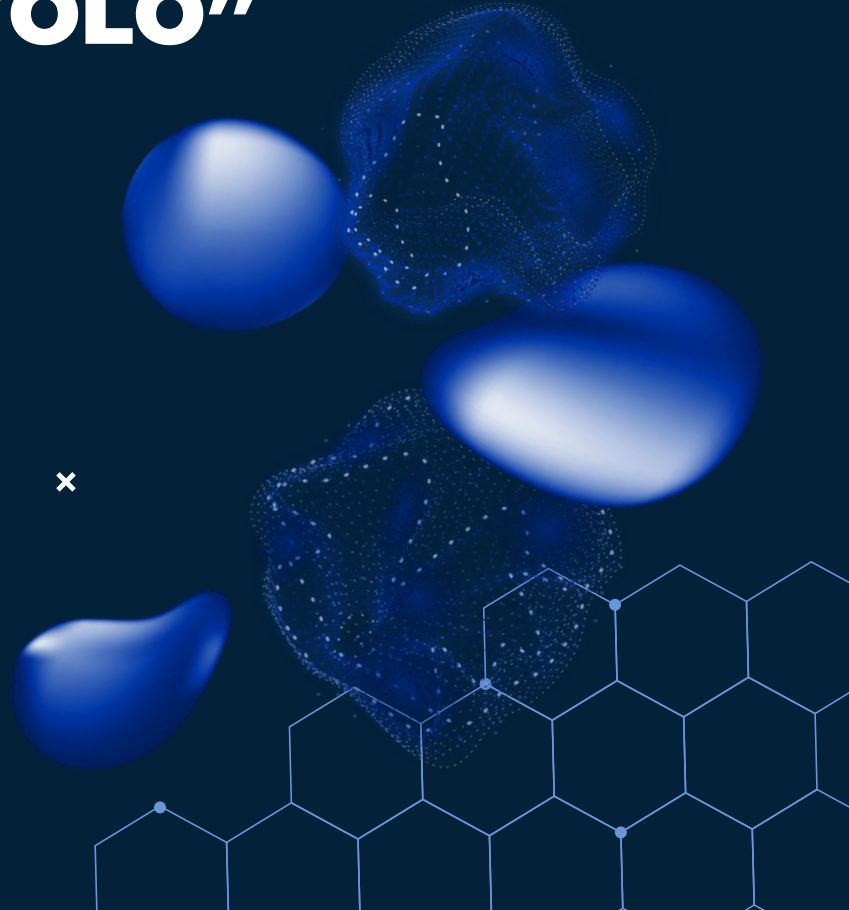
Was ist "YOLO"

„You Only Look Once“

- Deep-Learning-Verfahren
- Objekterkennung in Bilder und Videos
- Neuronales Netzwerk lokalisiert und klassifiziert Objekte

×

×





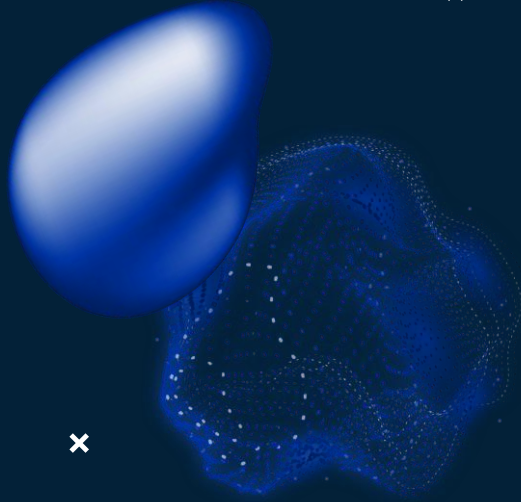
03

Methodisches Vorgehen

Erläuterung von Aufbau, Training und Funktionsweise des Systems

Hardware

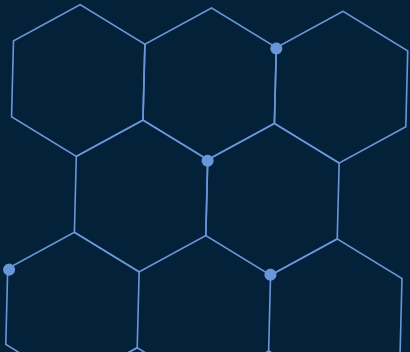
x



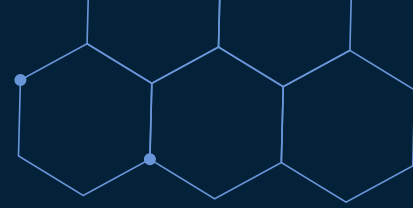
x

- Erstellt in SolidWorks
- Mit „Acrylnitril-Butadien-Styrol“ gedruckt
- physische Basis des Projekts
- Kameras, Beleuchtung

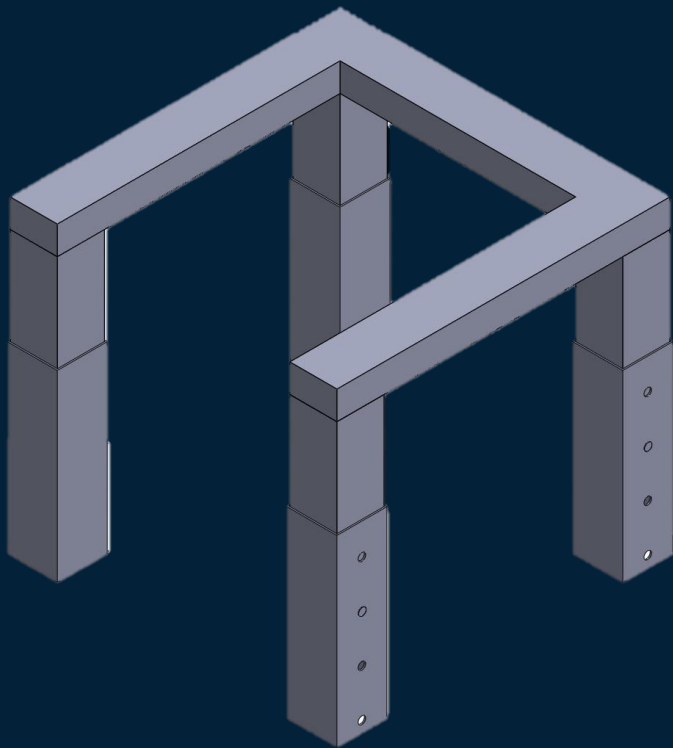
x



x



x x



x



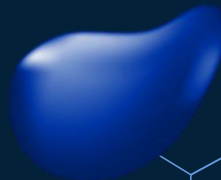
x



Besteht aus:

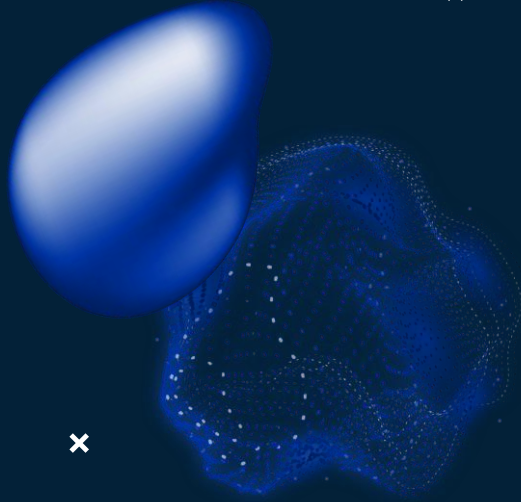
- 4 Höhenverstellbare Beine
- U-Stück

x



Software

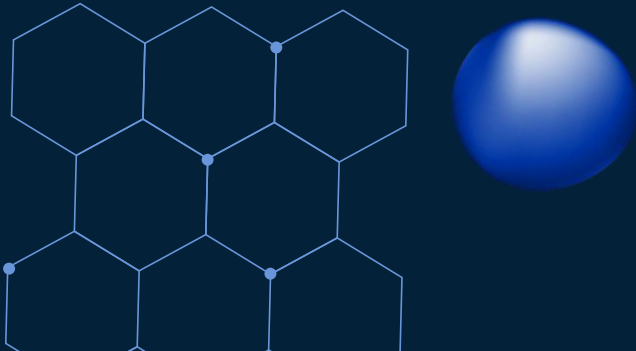
x



x

- Schnittstellen zu Kameras / Hardware
- Programmiersprache: Python, OpenCV, PyTorch
- Benutzeroberfläche / Visualisierung der Ergebnisse
- Automatisierte Klassifizierung von Fehlerarten

x

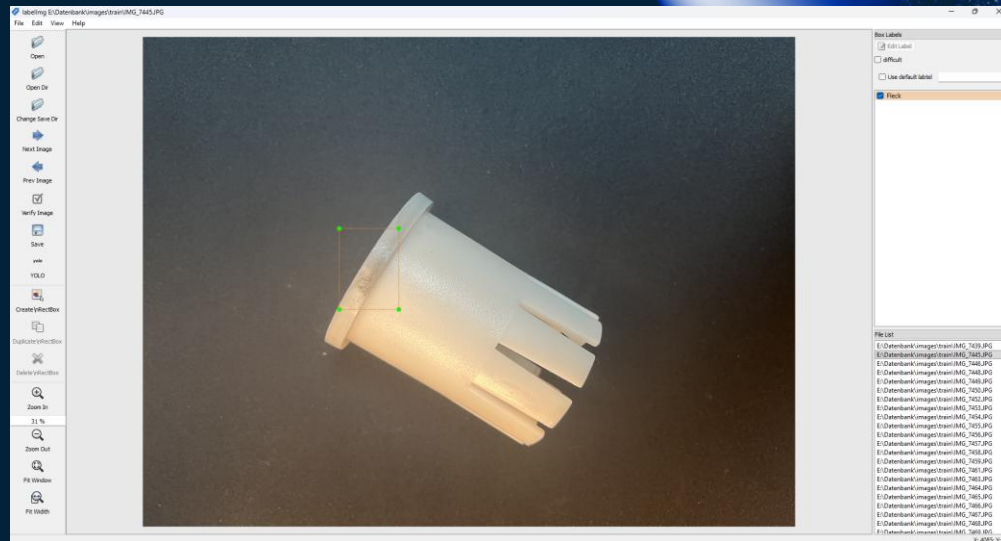


x

Training des Modells

×

- „Labellmg“ zur Annotierung
- Box um Fehler ziehen
- ×
- Datenbank/
 - └─ images/
 - └─ train/ (70%)
 - └─ val/ (30%)
 - └─ labels/
 - └─ train/ (70%)
 - └─ val/ (30%)



sort.py

```
sort.py > ...
1 import os #Interaktion mit dem Betriebssystem
2 import shutil #Verschieben von Dateien
3 import random #Mischen der Daten
4
5 base_path = r"E:\Datenbank" #Pfad zu der Datenbank
6 images_path = os.path.join(base_path, "images") #Pfad zu den Bildern
7 labels_path = os.path.join(base_path, "labels") #Pfad zu den Label
8
9 for split in ["train", "val"]:
10     os.makedirs(os.path.join(images_path, split), exist_ok=True) #Ordner Erstellen, exist_ok=True damit nicht Abturrz
11     os.makedirs(os.path.join(labels_path, split), exist_ok=True)
12
13 images = [
14     f for f in os.listdir(images_path)
15     if f.lower().endswith((".JPG", ".jpeg", ".jpg"))
16 ]
17
18 print(f"Gefundene Bilder: {len(images)}") # Rückmeldung fall keine Bilder gefunden wurden
19 if len(images) == 0:
20     print("Keine Bilder gefunden! Prüfe den Pfad oder die Dateieindungen.")
21     exit()
22
23 random.shuffle(images) #Mischt die Liste der Bildnamen
24
25 split_idx = int(0.7 * len(images)) #Gewünschte Größe der Trainingsbilder
26 train_images = images[:split_idx] #Ersten 70% erhalten die Trainingsbilder
27 val_images = images[split_idx:] #Erhalten die restlichen 30%
28
29 print(f"Train: {len(train_images)} Bilder")
30 print(f"Val: {len(val_images)} Bilder")
31
32 for img in train_images: #Verschiebung der Trainingsbilder
33     label = os.path.splitext(img)[0] + ".txt"
34     src_img = os.path.join(images_path, img)
35     src_label = os.path.join(labels_path, label)
36
37     if os.path.exists(src_img) and os.path.exists(src_label): #Sortiert wie Bilder und Label nach "train"
38         shutil.move(src_img, os.path.join(images_path, "train", img))
39         shutil.move(src_label, os.path.join(labels_path, "train", label))
40
41 for img in val_images:
42     label = os.path.splitext(img)[0] + ".txt"
43     src_img = os.path.join(images_path, img)
44     src_label = os.path.join(labels_path, label)
45
46     if os.path.exists(src_img) and os.path.exists(src_label): #Sortiert nach "val"
47         shutil.move(src_img, os.path.join(images_path, "val", img))
48         shutil.move(src_label, os.path.join(labels_path, "val", label))
49
50 print("Aufteilung abgeschlossen!")
```

- Automatische Verteilung von rund 2.000 Bild- und Textdateien
- 70% nach „train“
- 30% nach „val“



data.yaml

```
! data.yaml
1 train: E:\Datenbank\images\train #Pfad zu den Trainingsbilder
2 val: E:\Datenbank\images\val #Pfad zu den Validierungsbilder
3
4 nc: 4 #Anzahl der Klassen
5 names: ['Kratzer', 'fehlendes Material', 'Fleck', 'Anguss noch dran'] #Namen der Klassen
```

train_model.py

```
train_model.py > ...
1 from ultralytics import YOLO #import von YOLO
2
3 model = YOLO("yolov8n.pt") #Pfad zur zu trainierenden KI
4
5 model.train(
6     data = r"E:\Datenbank\data.yaml", #Pfad zur data.yaml
7     epochs = 50, #Anzahl der Durchläufe
8     imgsz = 640, #Bildgröße (Standard für YOLO)
9     augment = True #Trainingsgenauigkeit verbessern
10 )
11
12 print("Training abgeschlossen!") #Rückmeldung das Training abgeschlossen ist
```

main.py

```
main.py > ...
1 #Import aller benötigter Sachen
2 import cv2
3 import numpy as np
4 from ultralytics import YOLO
5
6 model = YOLO(r"E:\Seminararbeit\runs\detect\train20\weights\best.pt") #Pfad zur KI
7
8 model.model.names = {0: 'Kratzer', 1: 'fehlendes Material', 2: 'Fleck', 3: 'Anguss noch dran'} #Beschreibung der Klassen
9
10 cap1 = cv2.VideoCapture(0) #Initialisierung der Kameras
11 cap2 = cv2.VideoCapture(1)
12
13 while True:
14     ret1, img1 = cap1.read()
15     ret2, img2 = cap2.read()
16
17     if not ret1 or not ret2:
18         break
19
20     TARGET_HEIGHT = 480
21
22     img1 = cv2.resize(img1, (int(img1.shape[1] * TARGET_HEIGHT / img1.shape[0]), TARGET_HEIGHT))
23     img2 = cv2.resize(img2, (int(img2.shape[1] * TARGET_HEIGHT / img2.shape[0]), TARGET_HEIGHT))
24
25     # YOLO Inferenz
26     results1 = model(img1, conf=0.25)
27     results2 = model(img2, conf=0.25)
28
29     # Bounding Boxes einzeichnen
30     annotated1 = results1[0].plot()
31     annotated2 = results2[0].plot()
32
33     # Zusammenfügen
34     combined = np.hstack((annotated1, annotated2))
35
36     cv2.imshow("Beide Webcams", combined)
37
38     if cv2.waitKey(1) & 0xFF == ord("q"):
39         break
40
41 cap1.release()
42 cap2.release()
43 cv2.destroyAllWindows()
44
45
46 #Gibt die Kameras nach Beenden des Programms wieder frei
47 cap1.release()
48 cap2.release()
49 cv2.destroyAllWindows()
```

×

×

04

Probleme

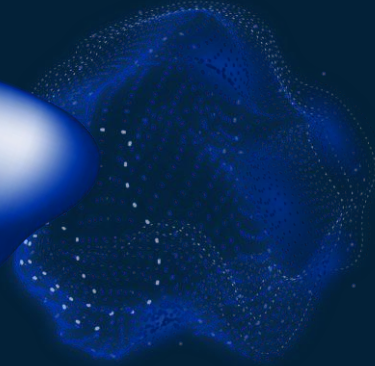
Inwieweit eignet sich ein mobiles Bildverarbeitungssystem zur automatisierten Fehlererkennung in der Produktion?



+



x

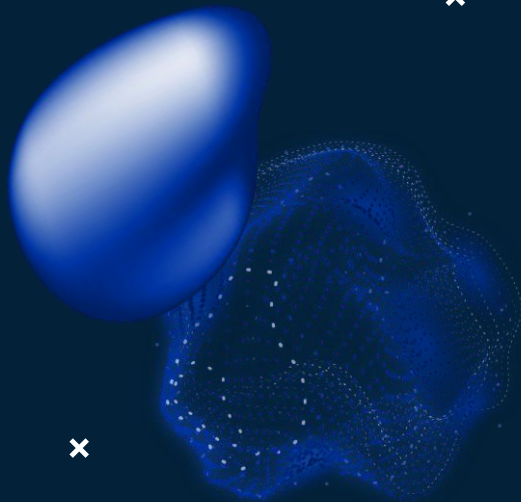


x

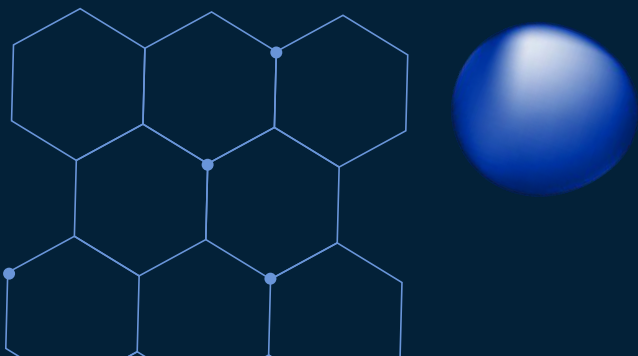


Erfassungsgrenzen

×



×



Datenerfassung

- begrenzte Anzahl an Fehlerbeispielen
- zeitintensive Bildaufnahme

×

Einfluss äußerer Faktoren

- Beleuchtung
- Kamerawinkel
- Hintergrund

×



Technische Limits

Training des Modells

- Abhängigkeit von der Datenqualität
- mögliche Über- oder Unteranpassung

Technische Grenzen

- begrenzte Rechenleistung
- Trainingsdauer

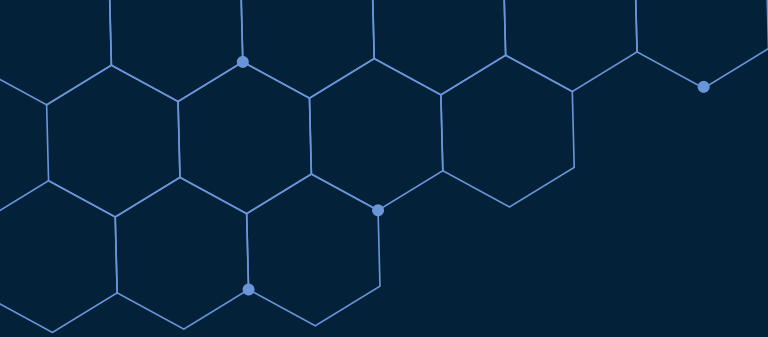


05

Ergebnisse

Analyse und Bewertung der erzielten Ergebnisse

Vorstellung



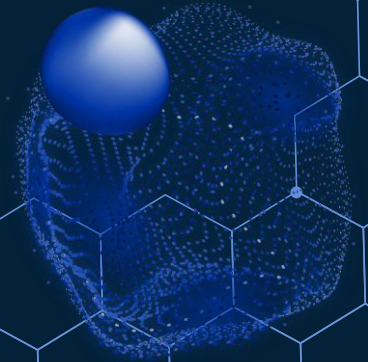
x



x



x



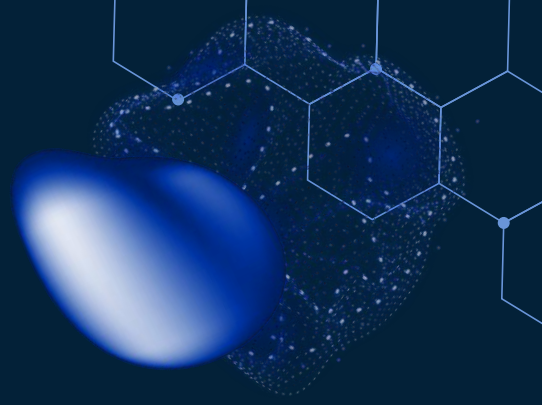
Ausblick

- Erweiterung des Datensatzes
- Maschine Learning implementieren
- Echtzeitprüfung während der Produktion
- Erkennung zusätzlicher Fehlerarten

x



x

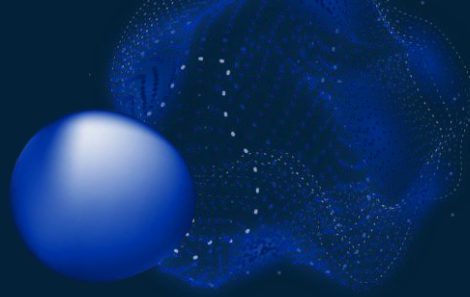


x





x



x

**Vielen Dank für die
Aufmerksamkeit!**



x

